

Practice Questions

From the past exam papers

Consider the following code fragment which "folds" a linear array xa.

```
const int MAXSIZE = 20000;
double fold, xa[MAXSIZE];
...
for ( i = 0; i < MAXSIZE/2; ++i )
    fold += xa[i] * xa[MAXSIZE - 1 - i];
```

branch.

Instruction	Latency	Repeat rate
Integer add/sub/logical/branch	1	1
Integer load/store	2	1
Floating-point load/store	3	1
Floating-point add/sub/multiply	2	1
Floating-point <u>multiply-add</u>	3	1
Floating-point division	19	21

no store, store at register

This code is to be executed on a MIPS R10K superscalar processor with 4 functional units capable of doing one floating-point operation and one integer operation per clock cycle. One of these operations could be a memory access (a load or a store). The integer operation could be a branch.

1. Write down the instructions involved in the execution of the loop, and the instruction schedule for one iteration. Give the flops per cycle for the schedule, and compute the functional unit utilization.
2. Unroll the loop once, and schedule the instructions. Give the flops per cycle for the schedule, and compute the functional unit utilization.
3. Unroll the loop twice, and schedule the instructions. Give the flops per cycle for the schedule, and compute the functional unit utilization.

We studied software pipelining in the context of the MIPS R10K processor, which is capable of performing two floating-point operations (one of which could be a memory access) and two integer operations (one of which could be a branch) per clock cycle.

You are asked to consider the following kernel, named DAXMY:

```
for (int i = 0; i < N; i++) {  
    y[i] = a * x[i] - y[i];  
}
```

pointer
multiply *load* *add.* *load.* *branch.*

Instruction	Latency	Repeat rate
Integer add/sub/logical/branch	1	1
Integer load/store	2	1
Floating-point load/store	3	1
Floating-point add/sub/multiply	2	1
Floating-point multiply-add	3	1
Floating-point division	19	21

Provide a schedule that maximizes the functional unit utilization. To this end, you may need to unroll the loop as required. Calculate the functional unit utilization you achieved. To earn full marks, your schedule should use the minimum number of instructions and achieve the maximum possible functional unit utilization.

You may assume reasonable mnemonics for any new instructions (i.e., instructions that we did not use in DAXPY) that you may need to use in your solution to DAXMY.

```
for ( j = 1; j <= N; j += 1 ) {  
    Ta: x[j] = x[j-1] + 10;  
    Tb: y[j] = x[j] + y[j];  
}
```

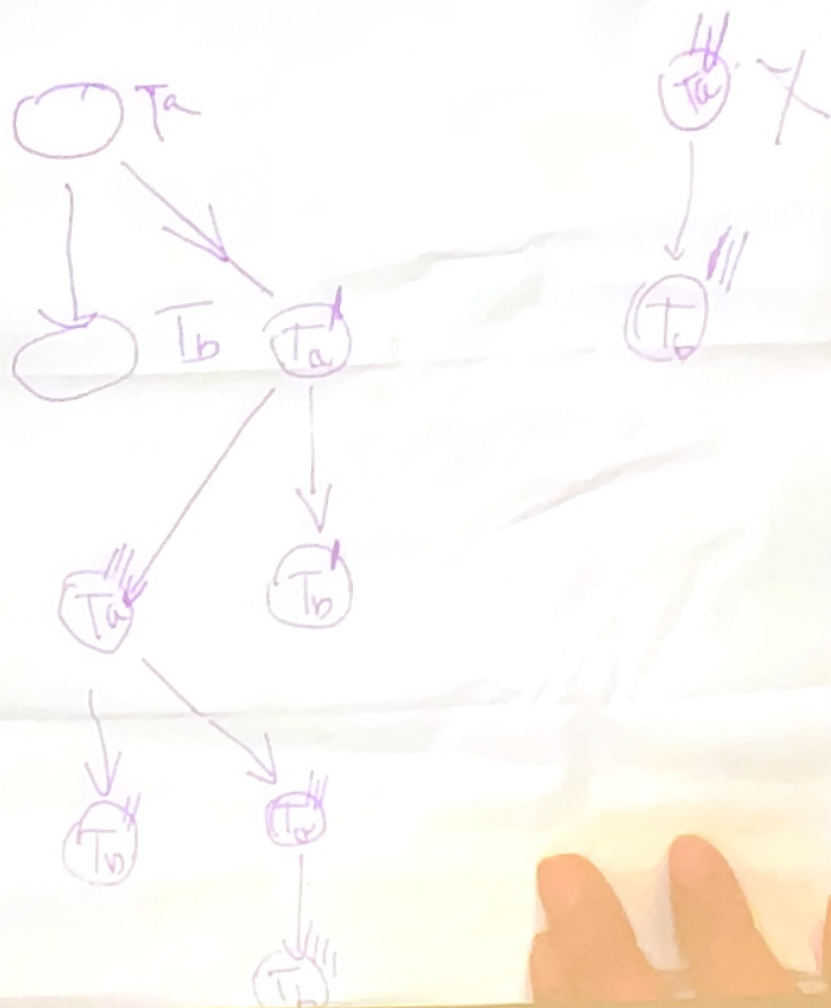
不能同时执行 T_a T_b.

Each statement in the loop body can be executed with a single instruction taking a clock cycle. The processor is superscalar with 4 homogenous functional units. Assume no branching overhead. How many cycles are needed to execute the entire loop?

How many cycles are needed to execute the loop unrolled once?

If the original loop is unrolled u times (i.e., the original instructions of the loop iteration are repeated u times in addition to the original), how many cycles are required to execute the unrolled loop? Show that the speedup of the loop unrolled u times is $(2u + 2)/(u + 2)$.

Show that the functional unit utilization for this loop, gained from unrolling, can never be greater than $1/2$.



问题

多个任务同时

分段...

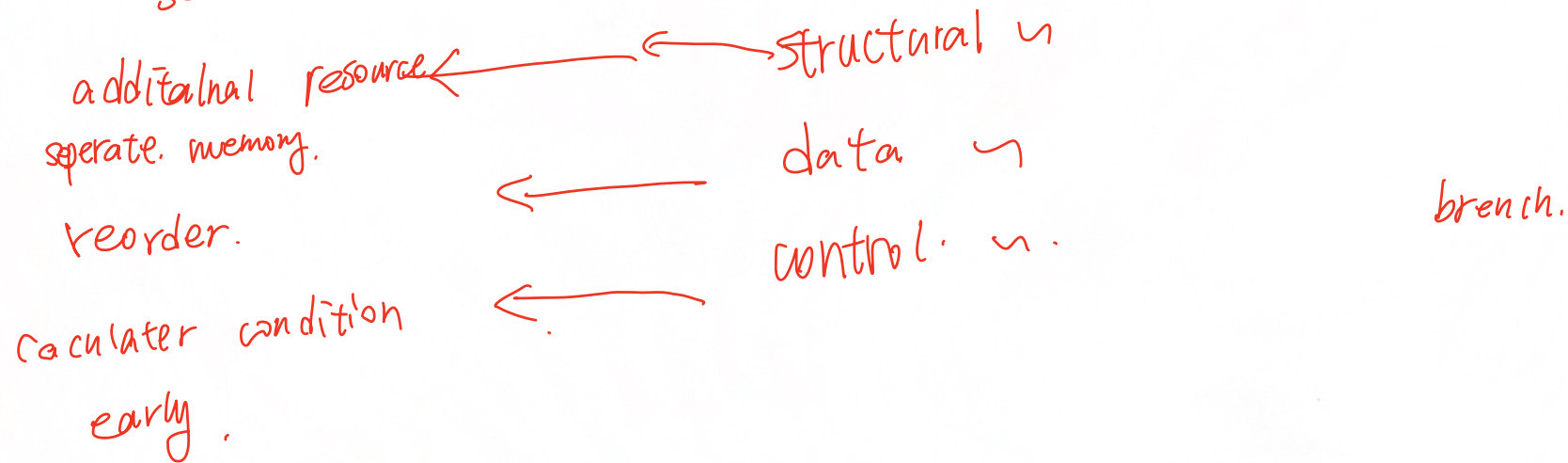
Compare and contrast parallel computation and pipeline computation in the light of speedup and resource requirements.

✓ 考

危害.

Explain the various pipeline hazards and how these hazards affect the performance of the pipeline. How do we reduce these hazards?

solution.



we can.
① stop pipeline, wait, but not good

Cache misses are categorized as compulsory misses, conflict misses, capacity misses, and coherency misses. Briefly describe each of these categories and discuss strategies to minimize coherency misses. What effects, if any, your strategies to minimize coherency misses would have on other miss types?

first time access to location. two thing go to same place.
space is not enough
reduce cache line size.

Someone invalidated it.

Briefly describe the cache coherence problem. Describe the MSI protocol and how it solves the cache coherence problem. What are the performance drawbacks of the MSI protocol? Describe how the MOSI and MESI protocols mitigate some of these performance issues. Highlight the differences between the three protocols, paying particular attention to the performance issues.

①. modified copy $\rightarrow$ read $\rightarrow$ need write back
memory

too many write back.

②. pick up some copy from memory, no one have copy.

no special state to show clean. share
write ... invalidate

— — —
No one needs
— — —

Describe schemes to reduce coherency misses, and discuss how each of these schemes would affect compulsory, capacity, and conflict misses.

Consider the following code fragment:

```
01  float sum[8]; float a[1024][16];
02  int i, j;
03  ....
04
05  for ( int i = 0; i < 16; ++i ) {
06      for ( int j = 0; j < 1024; ++j ) {
07          sum[i] = a[j][i];
08      }
09  }
```

8 processor. --

define 8 variable.

This code is to be executed in parallel on a 32 processor SMP system by assigning each of the outer loop iteration to a separate processor. Assuming that sizeof(float) is 4 and the cache line size is 64 bytes, what performance problems the parallel execution may face? How can you fix these problems? Re-write the code fragment with the problems fixed.

hard to have solution.